

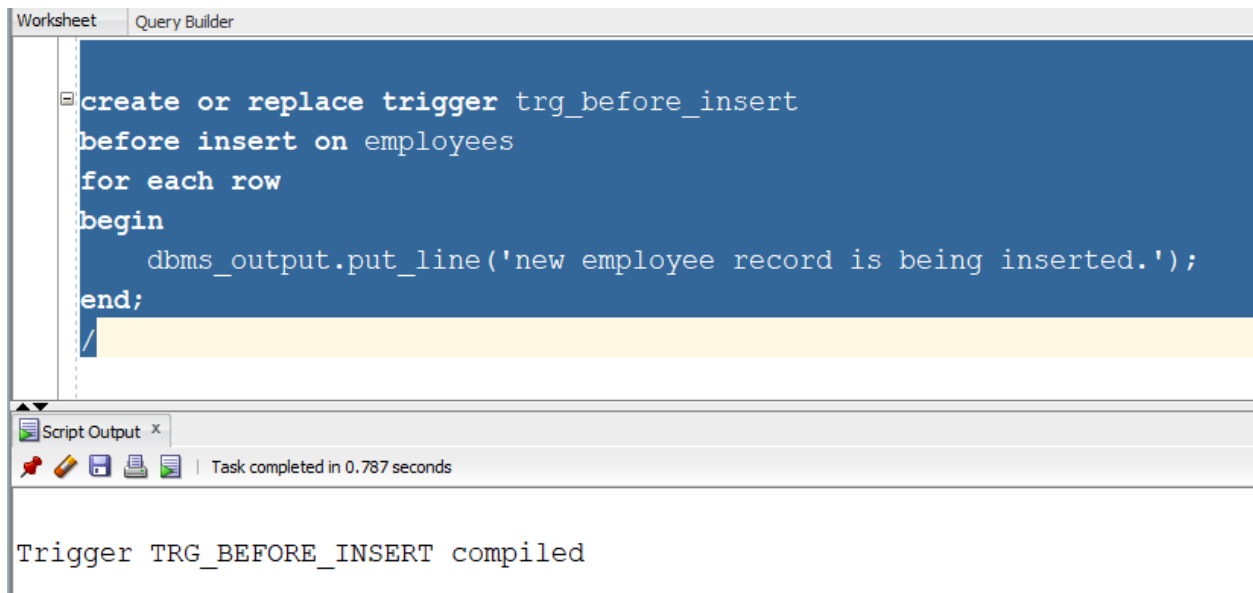
LAB 08

Yahya Shamim

24k-0020

Task 1

(a) Create a **BEFORE INSERT trigger** that displays a message when a record is inserted.



The screenshot shows a database query editor with a 'Worksheet' and 'Query Builder' tab. The main area contains a SQL script to create a trigger. Below the script, a 'Script Output' window shows the message 'Trigger TRG_BEFORE_INSERT compiled' and a status bar indicating 'Task completed in 0.787 seconds'.

```
create or replace trigger trg_before_insert
before insert on employees
for each row
begin
    dbms_output.put_line('new employee record is being inserted.');
```

end;

Script Output x

Task completed in 0.787 seconds

Trigger TRG_BEFORE_INSERT compiled

(b) Create a **BEFORE UPDATE trigger** that prints username.

```
create or replace trigger trg_before_update
before update on employees
for each row
begin
    dbms_output.put_line('update by user: ' || user);
end;
/
```

Script Output x

Task completed in 0.088 seconds

Trigger TRG_BEFORE_UPDATE compiled

(c) Create a trigger that prevents salary update if salary > 20000.

```
create or replace trigger trg_salary_guard
before update of salary on employees
for each row
begin
    if :new.salary > 20000 then
        raise_application_error(-20001,
            'salary cannot exceed 20000.');
```

Script Output x

Task completed in 0.076 seconds

Trigger TRG_SALARY_GUARD compiled

(d) Create a trigger that logs deleted employee records into another table.

```
create table deleted_employees_log (  
    employee_id number,  
    first_name varchar2(50),  
    last_name varchar2(50),  
    salary number,  
    deleted_at date  
);  
  
create or replace trigger trg_log_delete  
after delete on employees  
for each row  
begin  
    insert into deleted_employees_log  
    values (  
        :old.employee_id,  
        :old.first_name,  
        :old.last_name,  
        :old.salary,  
        sysdate  
    );  
end;  
/
```

Script Output x

Task completed in 0.146 seconds

Table DELETED_EMPLOYEES_LOG created.

Trigger TRG_LOG_DELETE compiled

Task 2

(a) Whenever a transaction is inserted, automatically store audit data (user + time).

```
create table transaction (  
    transaction_id number primary key,  
    transaction_date date,  
    created_at date,  
    created_by varchar2(50)  
);  
  
create or replace trigger trg_trans_audit  
before insert on transaction  
for each row  
begin  
    :new.created_by := user;  
    :new.created_at := sysdate;  
end;  
/  

```

Script Output x

Task completed in 0.095 seconds

Trigger TRG_TRANS_AUDIT compiled

(b) Create a **DDL trigger** that logs all table creation.

```
create table ddl_log (  
    object_name varchar2(50),  
    created_by varchar2(50),  
    created_at date  
);  
  
create or replace trigger trg_table_create_log  
after create on schema  
begin  
    insert into ddl_log (  
        object_name,  
        created_by,  
        created_at  
    )  
    values (  
        ora_dict_obj_name,  
        user,  
        sysdate  
    );  
end;  
/
```

Script Output x

Task completed in 0.138 seconds

Table DDL_LOG created.

Trigger TRG_TABLE_CREATE_LOG compiled

(c) Prevent **DROP TABLE** operation on critical tables.

```
create or replace trigger trg_prevent_drop
before drop on schema
begin
    if ora_dict_obj_type = 'TABLE' then
        raise_application_error(-20002,
            'dropping tables is not allowed.');
```

```
end if;
end;
/
```

Script Output x

Task completed in 0.08 seconds

Trigger TRG_PREVENT_DROP compiled

Task 3

Create a **LOGON trigger** that stores login time.

```

create table login_log (
    username varchar2(50),
    login_time date
);

create or replace trigger trg_logon_audit
after logon on schema
begin
    insert into login_log (
        username,
        login_time
    )
    values (
        user,
        sysdate
    );
end;
/

```

---q4---

Script Output x

Task completed in 0.234 seconds

Table LOGIN_LOG created.

Trigger TRG_LOGON_AUDIT compiled

Task 4

Create a system where:

- INSERT/UPDATE/DELETE triggers log data
- Uses **:NEW** and **:OLD**
- Maintain audit table


```
create or replace trigger trg_emp_full_audit
after insert or update or delete on employees
for each row
declare
    v_operation varchar2(10);
begin
    if inserting then
        v_operation := 'INSERT';
    elsif updating then
        v_operation := 'UPDATE';
    elsif deleting then
        v_operation := 'DELETE';
    end if;

    insert into emp_audit
        (operation, employee_id,
         old_salary, new_salary,
         old_job,    new_job,
         changed_by, changed_at)
    values
        (v_operation,
         coalesce(:new.employee_id, :old.employee_id),
         :old.salary,  :new.salary,
         :old.job_id,  :new.job_id,
         user, systimestamp);
end;
```

Script Output x

Task completed in 0.117 seconds

Trigger TRG_EMP_FULL_AUDIT compiled

Task 5

Design system with:

1. **DML trigger** → maintain backup table

```
create or replace trigger trg_emp_backup
after insert or update or delete on employees
for each row
begin
    insert into emp_backup
    values (
        nvl(:new.employee_id, :old.employee_id),
        nvl(:new.first_name, :old.first_name),
        nvl(:new.salary, :old.salary),
        ora_sysevent,
        sysdate
    );
end;
/
```

Script Output x

Task completed in 0.174 seconds

Table EMP_BACKUP created.

Trigger TRG_EMP_BACKUP compiled

2. **DDL trigger** → log schema changes

```
create table ddl_log (  
    obj_name varchar2(50),  
    action varchar2(20),  
    time date  
);  
  
create or replace trigger trg_ddl  
after create or drop or alter on schema  
begin  
    insert into ddl_log values (  
        ora_dict_obj_name,  
        ora_sysevent,  
        sysdate  
    );  
end;  
/  
  
create or replace trigger trg_session_log
```

Script Output x

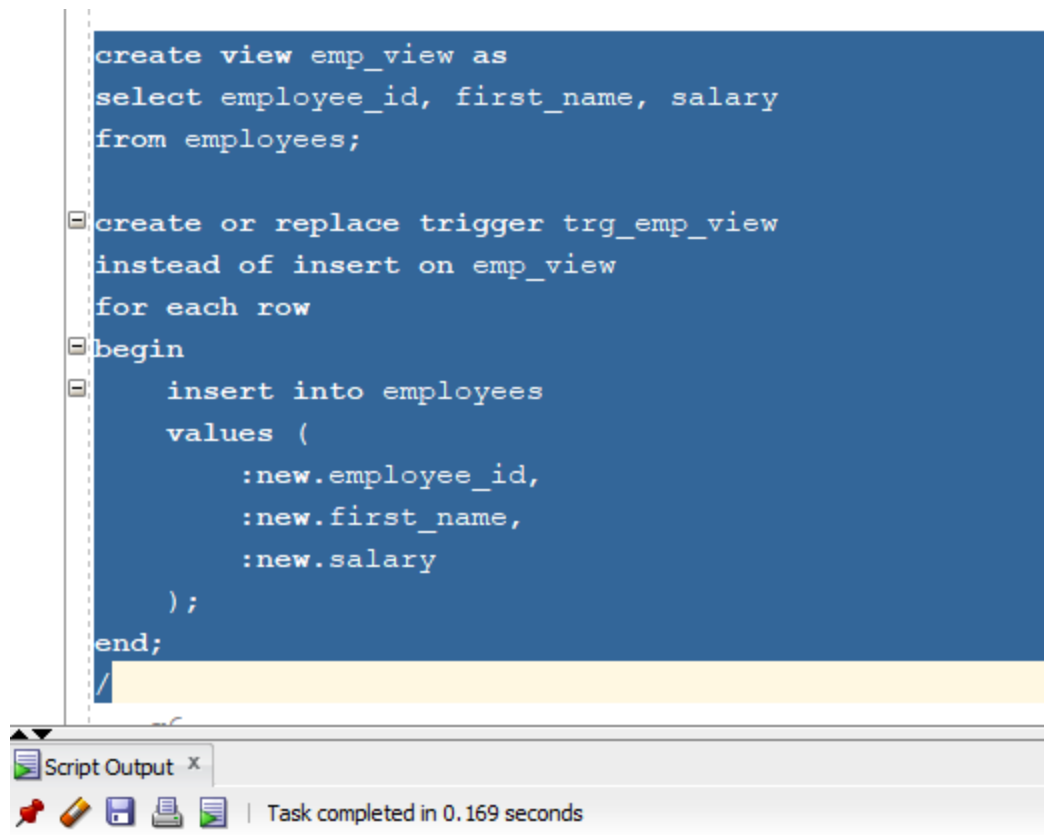
Task completed in 0.09 seconds

Trigger TRG_DDL compiled

3. INSTEAD OF trigger on a view

```
create view emp_view as
select employee_id, first_name, salary
from employees;

create or replace trigger trg_emp_view
instead of insert on emp_view
for each row
begin
insert into employees
values (
:new.employee_id,
:new.first_name,
:new.salary
);
end;
```



View EMP_VIEW created.

Trigger TRG_EMP_VIEW compiled

Task 6:

A large enterprise wants to implement a **data auditing and tracking system** for its employee database. Every time an employee record is inserted, updated, or deleted, the system must automatically record the old and new values, the user performing the operation, and the timestamp.

The auditing mechanism should work transparently without requiring manual intervention from developers or users. Additionally, the system should correctly handle different operations by distinguishing between inserted, deleted, and updated values.

Task:

Design and implement a trigger-based solution that:

- Automatically logs all INSERT, UPDATE, and DELETE operations
- Stores both old and new values where applicable
- Records user and timestamp information
- Uses appropriate trigger features to differentiate operations

```

create or replace trigger trg_employee_audit
after insert or update or delete on employees
for each row
begin
    if inserting then
        insert into employee_audit
        values (:new.employee_id, 'insert', null, :new.salary, user, sysdate);

    elsif updating then
        insert into employee_audit
        values (:old.employee_id, 'update', :old.salary, :new.salary, user, sysdate);

    elsif deleting then
        insert into employee_audit
        values (:old.employee_id, 'delete', :old.salary, null, user, sysdate);
    end if;
end;

```

Script Output x

Task completed in 0.107 seconds

Trigger TRG_EMPLOYEE_AUDIT compiled

Task 7:

A financial organization is developing a **high-security database system** where all database activities must be monitored and controlled. The system should maintain:

- A backup of critical tables that stays synchronized automatically
- A log of all schema-level changes such as CREATE, ALTER, and DROP
- Monitoring of user login and logout activities
- Controlled updates on complex views where direct modifications are not allowed

This system must ensure data integrity, enforce security policies, and provide full auditing capabilities across the database.

Task:

Design a complete trigger-based solution that:

- Synchronizes a backup table with the main table using DML triggers
- Logs schema changes using DDL triggers
- Tracks user activity using system/event triggers
- Enables controlled modifications on a non-updatable view using INSTEAD OF triggers

```
create or replace trigger trg_emp_backup
after insert or update or delete on employees
for each row
begin
    insert into employee_backup
    values (
        nvl(:new.employee_id, :old.employee_id),
        nvl(:new.first_name, :old.first_name),
        nvl(:new.salary, :old.salary),
        ora_sysevent,
        sysdate
    );
end;
```

Script Output x



Task completed in 0.097 seconds

Trigger TRG_EMP_BACKUP compiled

```
create or replace trigger trg_ddl_log
after create or alter or drop on schema
begin
    insert into ddl_log
    values (
        ora_dict_obj_name,
        ora_sysevent,
        sysdate
    );
end;
/
```

Script Output x



Task completed in 0.103 seconds

Trigger TRG_DDL_LOG compiled

```
create table user_activity (  
    username varchar2(50),  
    action varchar2(20),  
    log_time date  
);  
  
create or replace trigger trg_user_activ  
after logon on database  
begin  
    insert into user_activity  
    values (  
        user,  
        'logon',  
        sysdate  
    );  
end;  
/
```

Script Output x



Task completed in 0.139 seconds

Table USER_ACTIVITY created.

Error starting at line : 349 in command -
create or replace trigger trg_user_activity
after logon on database


```
create view emp_view as
select employee_id, first_name, salary
from employees;

create or replace trigger trg_emp_view
instead of update on emp_view
for each row
begin
    update employees
    set first_name = :new.first_name,
        salary = :new.salary
    where employee_id = :old.employee_id;
end;
```

Script Output x

Task completed in 0.153 seconds

ORA-00955: name is already used by an existing object

00955. 00000 - "name is already used by an existing object"

*Cause:

*Action:

Trigger TRG_EMP_VIEW compiled